

Name: Dwijesh Mistry

Class / Batch: TY4 – B

Roll No: 38

EXPERIMENT NO - 4

Aim: To implement the K-Means clustering algorithm using Python and visualize the clustered data.

Introduction: Clustering is an unsupervised learning technique used to group similar data points together.

K-Means is one of the most widely used clustering algorithms. It divides a dataset into K distinct

clusters based on the similarity of data points.

The algorithm works by iteratively:

1. Selecting initial centroids.
2. Assigning points to the nearest centroid.
3. Updating centroids as the mean of assigned points.
4. Repeating until centroids do not change.

This experiment demonstrates how K-Means can group data points and visualize clusters.

Procedure:

1. Import required libraries (random, math, matplotlib.pyplot).
2. Define the Euclidean distance function.
3. Define the K-Means function:
 - o Randomly select initial centroids.
 - o Assign each point to the nearest centroid.
 - o Update centroids based on cluster points.
 - o Repeat for a fixed number of iterations.
4. Prepare a sample dataset.
5. Call the K-Means function with $k = 2$ or 3 .
6. Print centroids and clusters.
7. Plot the data points with cluster colors and centroids.
8. Observe the clustering and verify results.

Program Code:

```
import random
import re
import os
import time
from typing import List, Optional, Tuple

def parse_int_list(text: str) -> List[int]:
    parts = [p for p in re.split(r"[\s,]+", text.strip()) if p]
    if not parts:
        raise ValueError("No numbers provided.")
    return [int(p) for p in parts]

def mean(values: List[int]) -> float:
    return sum(values) / len(values)

def assign_points_to_centroids(data: List[int], centroids: List[float]) -> List[int]:
    assignments: List[int] = []
    for x in data:
        best_idx = 0
        best_dist = abs(x - centroids[0])
        for i in range(1, len(centroids)):
            dist = abs(x - centroids[i])
            if dist < best_dist:
                best_dist = dist
                best_idx = i
        assignments.append(best_idx)
    return assignments

def build_clusters(data: List[int], assignments: List[int], k: int) -> List[List[int]]:
    clusters: List[List[int]] = [[] for _ in range(k)]
    for x, idx in zip(data, assignments):
        clusters[idx].append(x)
    return clusters

def reinit_empty_clusters(
    clusters: List[List[int]],
    centroids: List[float],
    rng: random.Random,
    data: List[int],
```

```

) -> Tuple[List[List[int]], List[float], bool]:
    changed = False
    for i, cluster in enumerate(clusters):
        if cluster:
            continue
        # If a cluster becomes empty, re-initialize its centroid to a random data
        point.
        centroids[i] = float(rng.choice(data))
        changed = True
    return clusters, centroids, changed

def kmeans_1d(
    data: List[int],
    k: int,
    *,
    seed: Optional[int] = None,
    max_iter: int = 100,
    verbose: bool = True,
) -> Tuple[List[List[int]], List[float]]:
    if k <= 0:
        raise ValueError("K must be a positive integer.")
    if k > len(data):
        raise ValueError("K cannot be greater than the number of input values.")
    if max_iter <= 0:
        raise ValueError("max_iter must be positive.")

    rng = random.Random(seed)
    # Pick K random starting centroids from the input numbers (integers).
    start_indices = rng.sample(range(len(data)), k)
    centroids = [float(data[i]) for i in start_indices]

    if verbose:
        print("Input data:", data)
        print(f"K = {k}")
        if seed is not None:
            print(f"Seed = {seed}")
        print("Initial centroids:", [int(c) if c.is_integer() else c for c in centroids])

    prev_assignments: Optional[List[int]] = None

```

```

for it in range(1, max_iter + 1):
    assignments = assign_points_to_centroids(data, centroids)
    clusters = build_clusters(data, assignments, k)
    clusters, centroids, reinit_happened = reinit_empty_clusters(clusters,
centroids, rng, data)
    if reinit_happened:
        assignments = assign_points_to_centroids(data, centroids)
        clusters = build_clusters(data, assignments, k)

new_centroids: List[float] = []
for i, cluster in enumerate(clusters):
    if cluster:
        new_centroids.append(mean(cluster))
    else:
        new_centroids.append(centroids[i])

if verbose:
    print(f"\nIteration {it}:")
    for i, cluster in enumerate(clusters):
        cluster_sorted = sorted(cluster)
        centroid_str = f"{new_centroids[i]:.4f}".rstrip("0").rstrip(".")
        print(f" Cluster {i + 1}: {cluster_sorted} | mean = {centroid_str}")

if prev_assignments is not None and assignments == prev_assignments:
    centroids = new_centroids
    if verbose:
        print("\nConverged: cluster sets unchanged.")
    return clusters, centroids

prev_assignments = assignments
centroids = new_centroids

if verbose:
    print("\nStopped: reached max_iter without full convergence.")
return clusters, centroids

def main() -> None:
    run_id = time.strftime("%Y-%m-%d %H:%M:%S")
    print(f"Run started ({run_id}) | PID={os.getpid()}")
    raw_numbers = input("Enter integers (space/comma-separated): ").strip()

```

```

data = parse_int_list(raw_numbers)

raw_k = input("Enter K (number of clusters): ").strip()
if not raw_k:
    raise ValueError("K is required.")
k = int(raw_k)

raw_seed = input("Optional seed (press Enter to skip): ").strip()
seed = int(raw_seed) if raw_seed else None

clusters, centroids = kmeans_1d(data, k, seed=seed, max_iter=100,
verbose=True)

print("\nFinal result:")
for i, cluster in enumerate(clusters):
    cluster_sorted = sorted(cluster)
    centroid_str = f"{centroids[i]:.4f}".rstrip("0").rstrip(".")
    print(f" Cluster {i + 1}: {cluster_sorted} | mean = {centroid_str}")

if __name__ == "__main__":
    main()

```

Output:

```

Enter integers (space/comma-separated): 1 2 3 5 6 7 10 11 12 13 14 15 20 22 23 24
Enter K (number of clusters): 3
Optional seed (press Enter to skip):
Input data: [1, 2, 3, 5, 6, 7, 10, 11, 12, 13, 14, 15, 20, 22, 23, 24]
K = 3
Initial centroids: [11, 12, 15]

Iteration 1:
Cluster 1: [1, 2, 3, 5, 6, 7, 10, 11] | mean = 5.625
Cluster 2: [12, 13] | mean = 12.5
Cluster 3: [14, 15, 20, 22, 23, 24] | mean = 19.6667

Iteration 2:
Cluster 1: [1, 2, 3, 5, 6, 7] | mean = 4
Cluster 2: [10, 11, 12, 13, 14, 15] | mean = 12.5
Cluster 3: [20, 22, 23, 24] | mean = 22.25

Iteration 3:
Cluster 1: [1, 2, 3, 5, 6, 7] | mean = 4
Cluster 2: [10, 11, 12, 13, 14, 15] | mean = 12.5
Cluster 3: [20, 22, 23, 24] | mean = 22.25

Converged: cluster sets unchanged.

Final result:
Cluster 1: [1, 2, 3, 5, 6, 7] | mean = 4
Cluster 2: [10, 11, 12, 13, 14, 15] | mean = 12.5
Cluster 3: [20, 22, 23, 24] | mean = 22.25

```

Conclusion: In this experiment, K-means clustering was implemented in Python for one-dimensional numerical data, where initial centroids were randomly selected and data points were iteratively assigned to the nearest centroid using absolute distance, followed by centroid updates based on cluster means. The algorithm continues until convergence or a maximum number of iterations is reached, effectively minimizing intra-cluster variance. The implementation also handles empty clusters to ensure stability, demonstrating that K-means is an efficient method for grouping data, though its effectiveness depends on proper selection of the number of clusters K and the initial centroids.

Review Questions:

1. What is the K-means clustering algorithm, and how does it work?

Ans: K-means is an unsupervised machine learning algorithm used to partition a dataset into K distinct, non-overlapping clusters. Each cluster is represented by a centroid, which is the mean of all data points in that cluster.

The algorithm works as follows:

1. **Initialization:** Randomly select K data points as initial centroids.
2. **Assignment Step:** Assign each data point to the nearest centroid based on a distance metric (absolute distance in this experiment).
3. **Update Step:** Recalculate each centroid as the mean of the assigned data points.
4. **Iteration:** Repeat the assignment and update steps until centroids stabilize or cluster assignments no longer change.

K-means aims to minimize the within-cluster sum of squared distances, making clusters as compact as possible.

2. How do you determine the optimal number of clusters in K-means

Ans: The optimal number of clusters K can be determined using several methods:

- **Elbow Method:** Plot the within-cluster sum of squares (WCSS) for different values of K . The optimal K is where the decrease in WCSS begins to level off (the “elbow” point).

- **Silhouette Score:** Measures how similar a data point is to its own cluster compared to other clusters. A higher average silhouette score indicates better clustering.
- **Domain Knowledge:** Prior understanding of the dataset may suggest a meaningful number of clusters.
- **Gap Statistic:** Compares clustering performance with that of random data to identify the best K .

Choosing an appropriate K is critical, as too few clusters may oversimplify the data, while too many clusters may lead to overfitting.

3. What are the common distance metrics used in Agglomerative Clustering?

Ans: Agglomerative clustering is a hierarchical clustering method that builds clusters by repeatedly merging the closest pairs. Common distance metrics include:

- **Euclidean Distance:** The straight-line distance between two points; most commonly used.
- **Manhattan Distance:** The sum of absolute differences between coordinates.
- **Cosine Distance:** Measures the cosine of the angle between two vectors; useful for text and high-dimensional data.
- **Chebyshev Distance:** The maximum difference along any coordinate dimension.

Additionally, agglomerative clustering uses **linkage criteria** to define inter-cluster distance:

- **Single Linkage:** Minimum distance between points in two clusters.
- **Complete Linkage:** Maximum distance between points.
- **Average Linkage:** Average distance between all points.
- **Ward's Method:** Minimizes variance within merged clusters.

GitHub Link: https://github.com/terminator0106/DWM_Practical